

DEVELOPER DOCUMENTATION · v1.0

NAAM DEKHO

नाम देखो

Production engineering reference — architecture, APIs, WebSockets, connectors

FOR THE NAAM DEKHO ENGINEERING TEAM

This document is the canonical engineering reference for building the Naam Dekho platform. It covers system architecture, REST API contracts, WebSocket protocol, scanner module specifications, the complete connector knowledge base (with URLs, pricing, and rate limits), authentication and billing flows, deployment topology, and security and compliance posture.

Paired deliverable

Naam_Dekho_Connector_Catalog.xlsx

Three-sheet workbook: Connector Catalog · Scanner Modules · Cost Model

Prepared: 14 May 2026 · Document version: 1.0 · Audience: Engineering, DevOps, Data, SRE

Table of Contents

In Microsoft Word: right-click the table below and choose "Update Field" to populate page numbers.

1. At a Glance

1.1 What we are building

Naam Dekho is a name-availability and name-suitability platform that accepts a single proposed name and returns, in a few seconds, the status of that name across up to sixty-two distinct sources. Two product modes are available to the user — a comprehensive Startup mode (full sixty-two checks) and a focused Baby mode (four checks: pronunciation, linguistic safety, Chaldean numerology, and social handle availability).

1.2 System pillars

- HTTPS REST API (FastAPI) for scan submission, account, billing, and PDF retrieval.
- Real-time WebSocket channel (Socket.IO with Redis adapter) for streaming per-tile results back to the client as they arrive.
- Asynchronous task queue (Redis-backed BullMQ) with priority lanes, retry policy, and per-source rate limiters.
- Six independent scanner worker pools, each owning a specific class of external sources.
- Two persistent data stores: PostgreSQL for users/scans/billing, and S3/R2 for generated PDF reports.
- Three caching layers: in-memory LRU per worker, shared Redis cache (TTL-tuned per source), and CDN edge cache for static assets.

1.3 Stack summary

Concern	Choice
Front-end	React 18 + Vite + Tailwind CSS; Socket.IO-client; Service Worker for PWA; deployed to Cloudflare Pages.
REST API	FastAPI on Python 3.12 with uvicorn-workers + gunicorn supervision. OpenAPI auto-generated.
WebSocket	Socket.IO 4 (Node.js 20) with @socket.io/redis-adapter for horizontal scaling across instances.
Task queue	BullMQ 5 (Node.js workers) on Redis 7 with priority lanes, exponential-backoff retries, and per-source token-bucket rate limiters.
Scanner workers	Mixed Python (Playwright-based scrapers) and Node.js (API-based connectors). Containerised with Docker; orchestrated by Kubernetes (EKS) or Hetzner Cloud (cheaper alternative).
Database	PostgreSQL 16 (managed: Neon or AWS RDS). Tables: users, scans, scan_results, billing_events, agency_leads.

Cache & pub/sub	Redis 7 (managed: Upstash or AWS ElastiCache). Used for cache, pub/sub, task queue.
Object storage	Cloudflare R2 (preferred — zero egress fees) or AWS S3 for PDF reports. Pre-signed URLs.
Authentication	Firebase Phone Auth (OTP) for free + retail tiers; bespoke JWT-on-top for agency API tier.
Payments	Razorpay (India) + Stripe (international); webhooks → <code>billing_events</code> table.
PDF generation	WeasyPrint (Python) for the corporate scan report; ReportLab for the baby keepsake (more typography control).
CAPTCHA solving	2Captcha (primary) + Anti-Captcha (failover) for deep-tier scans only.
Proxy network	Bright Data residential (premium sources) + Smartproxy (cost-effective for most). Rotating session-sticky.
Observability	Sentry (errors) + Grafana Loki (logs) + Prometheus (metrics) + Better Stack (uptime/alerts).
CI / CD	GitHub Actions → Docker registry → ArgoCD (k8s) or simpler GitHub-Actions-direct-deploy (Hetzner).
Infrastructure	Cloudflare for edge + DNS + R2 + Pages. Hetzner Cloud (Frankfurt + Singapore) for workers. AWS for managed PG/Redis.

2. Architecture

2.1 Layered system view

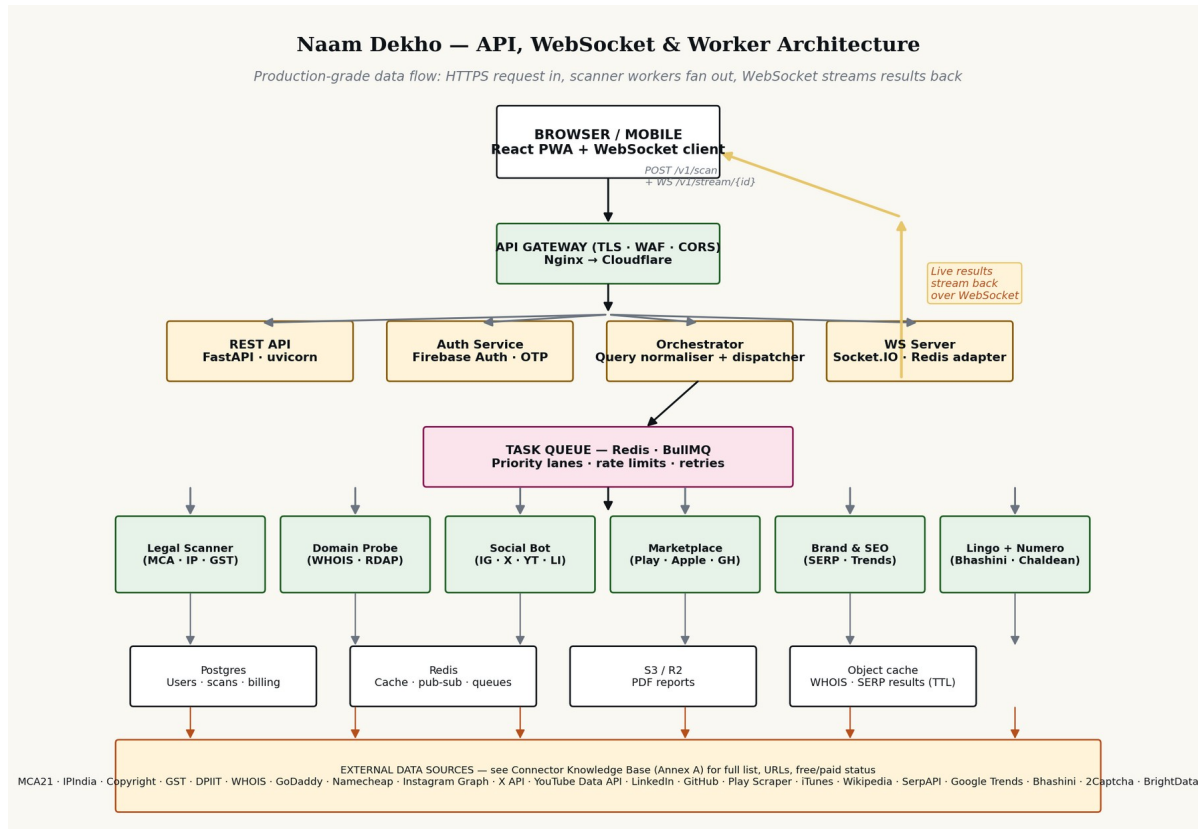


Figure 1 — Production system architecture — gateway, orchestrator, queue, scanner workers, data stores

2.2 WebSocket message lifecycle

Figure 2 traces a single scan request through the system. The same lifecycle applies to both Startup and Baby modes; the only difference is the number of platform queries fanned out at the dispatch step.

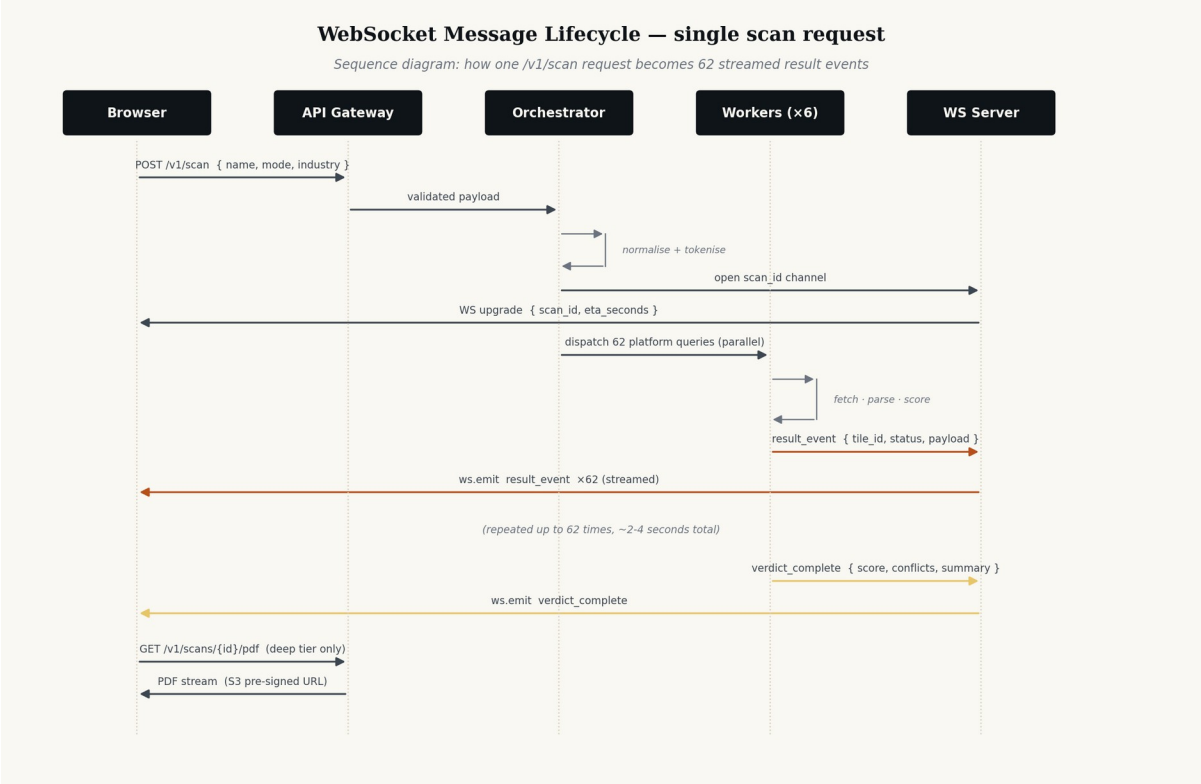


Figure 2 — Sequence diagram for one /v1/scan request

2.3 Original twelve-step pipeline (recap)

For completeness, Figure 3 reproduces the original twelve-step name-to-verdict pipeline. This pipeline is the central business-logic IP and is shared across all modes.

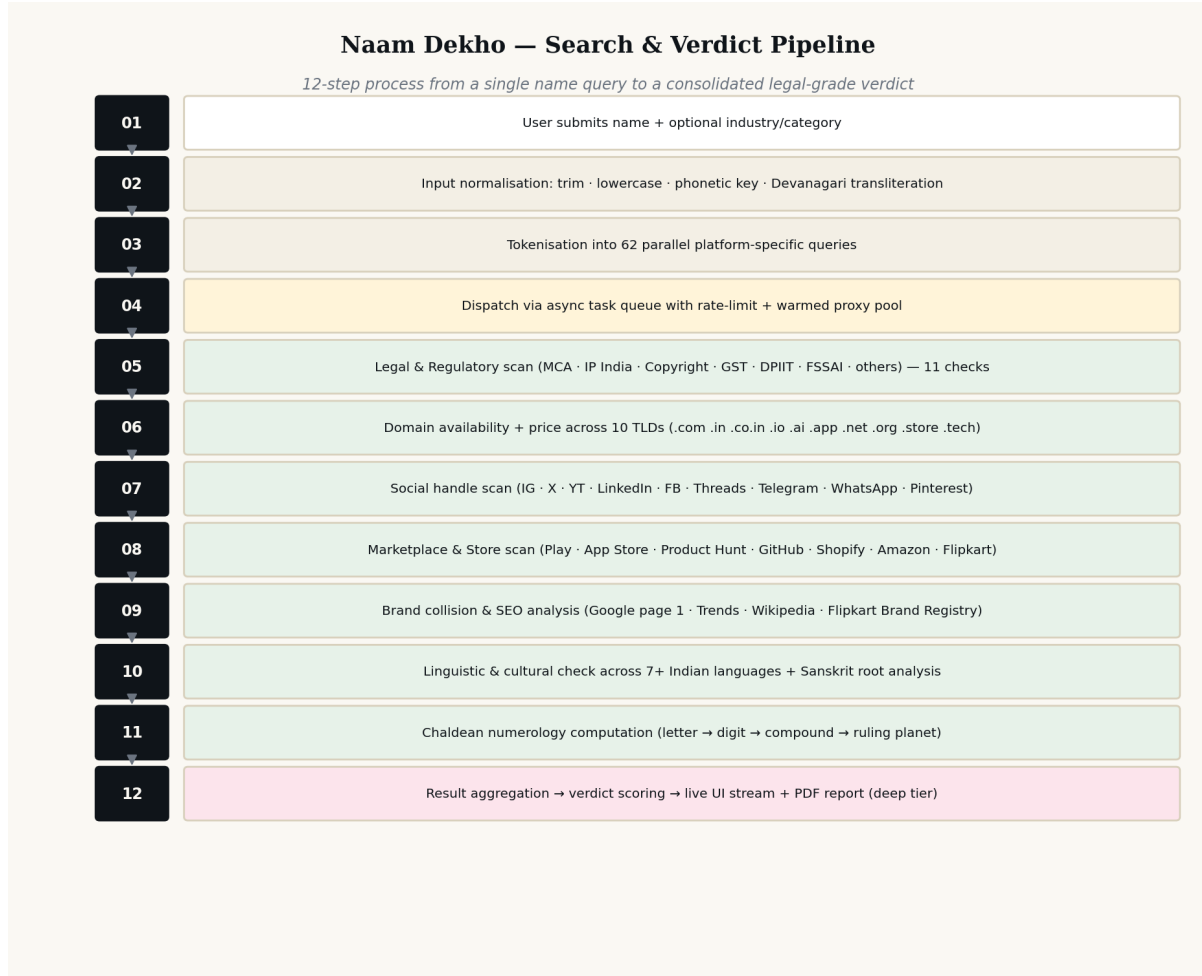


Figure 3 — Twelve-step pipeline — copy reproduced from the Copyright Filing

2.4 Deployment topology

Recommended primary topology for production:

- Cloudflare edge — DNS, WAF, DDoS, CDN, R2 (object storage), Pages (static front-end).
- Hetzner Cloud — six worker pools (one per scanner family) running on CCX21 instances (4 vCPU / 16 GB / dedicated CPU) in Frankfurt and Singapore for India-latency.
- AWS Mumbai — managed PostgreSQL (RDS) and managed Redis (ElastiCache) for India data-residency posture.
- API + WebSocket — three uvicorn pods + three Socket.IO pods, fronted by a Cloudflare Load Balancer. Each pod is stateless.
- Total bare cost at launch traffic (~50,000 scans/month) is approximately ₹35,000–₹50,000/month of cloud spend. Scales linearly thereafter.

3. REST API Contracts

All endpoints are versioned under /v1. Base URL: <https://api.naamdekho.in>. All requests/responses are JSON over HTTPS. Errors follow RFC 7807 (Problem Details for HTTP APIs).

3.1 Authentication

- Free tier: no auth required for /v1/scan submissions.
- Retail paid tier (₹49 deep, ₹29 keepsake, ₹99 shortlist): Firebase ID token in `Authorization: Bearer <token>` header.
- Agency tier: long-lived JWT issued from the dashboard; per-organisation quota enforced server-side via Redis token-bucket.

3.2 POST /v1/scan — start a new scan

Request body (JSON):

```
{
  "name": "Vyana",
  "mode": "business" | "baby",
  "industry": "saas-tech", // optional, sharpens TM class relevance
  "tier": "free" | "deep" | "agency",
  "callback_url": "https://...", // optional, webhook on completion
  "options": {
    "include_pending": true,
    "skip_categories": ["fssai"]
  }
}
```

Response (200 OK):

```
{
  "scan_id": "scn_01HXX...",
  "websocket_url": "wss://ws.naamdekho.in/v1/stream/scn_01HXX...",
  "eta_seconds": 4,
  "expires_at": "2026-05-14T10:30:00Z"
}
```

Errors:

- 400 — invalid name (empty, > 64 chars, contains forbidden tokens).
- 401 — missing/invalid auth (for paid tiers).
- 402 — payment required (agency over quota).
- 429 — rate-limited (free tier — 60 scans / IP / hour).
- 503 — orchestrator overloaded (retry with `Retry-After` header).

3.3 GET /v1/scans/{scan_id} — fetch consolidated result

Returns the full scan envelope once the scan is complete. Suitable for non-streaming clients.

```
{
  "scan_id": "scn_01HXX...",
  "name": "Vyana",
  "mode": "business",
  "status": "complete" | "running" | "failed",
  "verdict": {
    "score": 67,
    "clear": 42,
    "conflict": 13,
    "warn": 5,
    "pending": 2
  },
  "tiles": [
    { "tile_id": "mca", "category": "legal", "status": "ok", "summary": "...", "detail": {...} },
    { "tile_id": "tm_class_35", "category": "legal", "status": "no", "summary": "...", "detail": {...} },
    ...
  ],
  "started_at": "2026-05-14T10:25:14Z",
  "completed_at": "2026-05-14T10:25:18Z"
}
```

3.4 GET /v1/scans/{scan_id}/pdf — fetch report

Returns a 302 redirect to a pre-signed Cloudflare R2 URL valid for 15 minutes. Only available for deep, keepsake, shortlist, and agency tiers.

3.5 POST /v1/agency-leads — submit agency interest

Public endpoint. Body:

```
{
  "name": "Rohan Mehta",
  "role": "Founder",
  "company": "Naming House Pvt Ltd",
  "firm_type": "naming-agency",
  "email": "rohan@naminghouse.in",
  "phone": "+91 98765 43210",
  "expected_volume": "200-1000",
  "budget_range": "50000-200000",
  "notes": "..."
}
```

Response: 201 Created. Writes to agency_leads table and triggers a Slack alert to #sales.

3.6 POST /v1/auth/request-otp

Body: `{ "phone": "+919876543210" }`. Triggers Firebase Phone Auth OTP. Response: 200 OK with `request_id`.

3.7 POST /v1/auth/verify-otp

Body: `{ "request_id": "...", "code": "123456" }`. Response 200 with `{ id_token, refresh_token, user }`.

3.8 Other endpoints

- GET /v1/me — current user profile, scan history (paginated).
- POST /v1/billing/checkout — create Razorpay/Stripe payment intent.
- POST /v1/billing/webhook — provider webhook receiver (signature-verified).
- GET /v1/openapi.json — auto-generated OpenAPI 3.1 spec.
- GET /v1/healthz — liveness probe; GET /v1/readyz — readiness probe.

4. WebSocket Protocol

The WebSocket channel is the primary mechanism by which scan results are delivered to the client. The protocol is Socket.IO v4 over WSS, with Redis adapter for multi-node fan-out.

4.1 Connection

Client connects to `wss://ws.naamdekho.in/v1/stream/{scan_id}`. The `scan_id` is bound to the channel — clients cannot subscribe to `scan_ids` they did not initiate (enforced server-side via JWT or anonymous IP-binding for free tier).

4.2 Server → Client events

Event	Payload
scan_started	{ <code>scan_id</code> , <code>total_tiles</code> , <code>eta_seconds</code> } — sent immediately after connection.
result_event	{ <code>tile_id</code> , <code>category</code> , <code>status</code> , <code>summary</code> , <code>detail</code> , <code>latency_ms</code> } — one event per platform tile, streamed as they complete.
progress	{ <code>completed</code> , <code>total</code> } — periodic (every 500ms) for the progress bar.
hud_update	{ <code>verdict_score</code> , <code>clear</code> , <code>conflict</code> , <code>warn</code> , <code>pending</code> } — sent on each <code>result_event</code> so the four-card HUD updates live.
verdict_complete	{ <code>scan_id</code> , <code>score</code> , <code>total_time_ms</code> , <code>pdf_url?</code> } — sent when all tiles have completed.
tile_error	{ <code>tile_id</code> , <code>error_code</code> , <code>retry: bool</code> } — sent when a tile fails. Client renders "Checking..." with a small warning glyph; not a fatal scan error.
scan_failed	{ <code>reason</code> , <code>retryable</code> } — sent only on a fatal scan failure (orchestrator dead, queue overflow, etc.).

4.3 Client → Server events

- `subscribe` — re-attach to an existing `scan_id` (used on reconnect).
- `cancel` — abort an in-flight scan.
- `ping` — heartbeat (handled by Socket.IO transport).

4.4 Reconnection behaviour

- Client uses Socket.IO's built-in reconnect with exponential backoff (1s, 2s, 4s, max 30s).
- On reconnect, client emits ``subscribe`` with the `scan_id`; server replays any missed events from the Redis ring buffer (last 300 events / 5 minutes).

4.5 Backpressure & ordering

- Events are NOT guaranteed strictly ordered — tiles complete in independent threads. Each event carries a `tile\_id` so client can render in the correct grid cell.
- Server applies per-connection rate limiting at 200 events/second to protect slow clients.
- Client should debounce HUD re-renders to 60fps (16ms).

5. Scanner Module Specifications

Six scanner module families. Each runs in its own worker pool, has its own connector dependencies, its own cache TTLs, and its own owning squad. The detailed connector knowledge base is in §7 below.

5.1 Legal Scanner (11 + 1 fallback)

- Owned by: Legal-Data squad.
- Connectors: MCA21, IP India, Copyright, GST, DPIIT, FSSAI, RBI, SEBI, IRDAI, Patent Office, State S&E, Trademarkia (fallback).
- Technology: Python 3.12 + Playwright (headful Chromium for sources with anti-bot). Per-source warmed-proxy sessions.
- Worker pool size: 12 (scales to 30 under deep-tier load).
- Avg response time: 1.8–3.5s per source. Total Legal-scan time budget: 6s for free tier (parallel), 25s for deep tier (deeper crawling + class-wise TM).
- Cache TTL: 24h for register hits; 7d for misses (misses are sticky).
- CAPTCHA: 2Captcha for IP India, MCA on deep tier. Free tier skips CAPTCHA-gated deep checks.

5.2 Domain Probe (10)

- Owned by: Infra squad.
- Connectors: WHOIS (port 43), RDAP, GoDaddy API, Namecheap API, WhoisXML (fallback), Domainr, INRegistry, Cloudflare Registrar.
- Technology: Node.js 20 (async whois client + axios for REST). No browser needed.
- Worker pool size: 8.
- Avg response time: 300–900ms.
- Cache TTL: 5min for availability; 24h for pricing.
- Cost: minimal — almost entirely free APIs and registrar partner programmes.

5.3 Social Handle Bot (9)

- Owned by: Social-Integrations squad.
- Connectors: Instagram (URL probe), X v2 API, YouTube Data API v3, LinkedIn (URL probe), Facebook (URL probe), Threads (URL probe), Telegram Bot API, WhatsApp Business directory, Pinterest API.
- Technology: Node.js 20 + light Playwright when JS-rendered profile pages required.
- Worker pool size: 10.
- Avg response time: 400–1,200ms.

- Cache TTL: 30min for availability (handles are reserved quickly so freshness matters).

5.4 Marketplace Crawler (7)

- Owned by: Infra squad.
- Connectors: google-play-scraper (npm), iTunes Search API, Product Hunt v2 GraphQL, GitHub REST API, Shopify DNS probe, ScrapingBee (Amazon + Flipkart).
- Technology: Node.js 20 + Python.
- Worker pool size: 8.
- Avg response time: 600–2,200ms (Amazon scraping is the slowest).
- Cache TTL: 12h for app listings; 24h for sellers.

5.5 Brand & SEO Engine (8)

- Owned by: Brand-Intel squad.
- Connectors: Google CSE, SerpAPI (fallback), ScrapingBee (for hard pages), Bright Data (deep tier only), Wikipedia, Wikidata SPARQL, pytrends, Crunchbase Basic.
- Technology: Python 3.12 (httpx + asyncio).
- Worker pool size: 6 (Google CSE quota-bound).
- Avg response time: 1,200–3,000ms.
- Cache TTL: 6h for SERP; 24h for Wikipedia.

5.6 Linguistic + Numerology Engine (7)

- Owned by: NLP squad.
- Connectors: Bhashini (Govt of India), Indic NLP Library, Aksharamukha, Google Translate (fallback), ICU4J, custom Chaldean engine.
- Technology: Python 3.12, mostly local computation.
- Worker pool size: 4.
- Avg response time: 100–400ms (the fastest scanner).
- Cache TTL: permanent (deterministic given fixed input). Stored keyed by SHA-256 of normalised input.

6. Data Models

6.1 PostgreSQL schema (core tables)

```
-- users
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  phone       TEXT UNIQUE,
  email       TEXT UNIQUE,
  display_name TEXT,
  tier         TEXT NOT NULL CHECK (tier IN ('free','retail','founder-pro','agency')),
  agency_org_id UUID REFERENCES agencies(id),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  last_login_at TIMESTAMPTZ
);

-- scans
CREATE TABLE scans (
  id          TEXT PRIMARY KEY,      -- ULID, prefixed "scn_"
  user_id     UUID REFERENCES users(id),
  anonymous_ip INET,                 -- for free tier
  name        TEXT NOT NULL,
  name_normalised TEXT NOT NULL,
  mode        TEXT NOT NULL CHECK (mode IN ('business','baby')),
  industry    TEXT,
  tier         TEXT NOT NULL,
  status      TEXT NOT NULL,
  verdict_score INT,
  total_tiles INT NOT NULL,
  clear_count INT,
  conflict_count INT,
  started_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  completed_at TIMESTAMPTZ,
  pdf_object_key TEXT
);
CREATE INDEX idx_scans_user ON scans(user_id, started_at DESC);

-- per-tile results
CREATE TABLE scan_results (
  scan_id     TEXT REFERENCES scans(id) ON DELETE CASCADE,
  tile_id     TEXT NOT NULL,
  category    TEXT NOT NULL,
  status      TEXT NOT NULL,      -- ok|no|warn|pending|error
  summary     TEXT,
  detail      JSONB NOT NULL,
  latency_ms  INT,
  PRIMARY KEY (scan_id, tile_id)
);

-- billing events
CREATE TABLE billing_events (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

user_id      UUID REFERENCES users(id),
provider     TEXT NOT NULL,      -- razorpay|stripe
intent_id    TEXT NOT NULL,
amount_paise BIGINT NOT NULL,
currency     TEXT NOT NULL,
status       TEXT NOT NULL,
scan_id      TEXT REFERENCES scans(id),
created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
raw_payload  JSONB
);

-- agency leads
CREATE TABLE agency_leads (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT NOT NULL,
  role        TEXT,
  company     TEXT NOT NULL,
  firm_type   TEXT,
  email       TEXT NOT NULL,
  phone       TEXT NOT NULL,
  expected_volume TEXT,
  budget_range TEXT,
  notes       TEXT,
  status      TEXT NOT NULL DEFAULT 'new',
  assigned_to UUID,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

6.2 Redis key namespaces

- ``cache:source:{source_id}:{name_hash}`` — per-source result cache (TTL varies by source, see §7).
- ``queue:scan`` — BullMQ primary task queue.
- ``queue:scan:deep`` — BullMQ priority queue for paid deep scans.
- ``ratelimit:src:{source_id}`` — token-bucket counter.
- ``ws:scan:{scan_id}`` — ring buffer of last 300 WS events for reconnect replay.
- ``auth:otp:{phone}`` — OTP attempt counter (TTL 10min, max 5 attempts).

7. Connector Knowledge Base

This section is the single source of truth for every external connector. The same data is delivered as a fully formatted, sortable, filter-friendly workbook at [Naam_Dekho_Connector_Catalog.xlsx](#) (three sheets: Connector Catalog · Scanner Modules · Cost Model — the cost model has live formulas).

The catalogue below is grouped by scanner family. Connectors marked "Free" require no payment but may require account creation; connectors marked with a price are pay-as-you-go or subscription.

7.1 Legal & Regulatory

Connector	URL	Free / Paid	Auth	Notes
MCA21	mca.gov.in	Free (manual / scraping)	Session + CAPTCHA	Use Playwright
IP India TM	ipindiaonline.gov.in	Free (scraping)	Session + CAPTCHA	All 45 classes
Copyright Office	copyright.gov.in	Free	Session	Cache 7d
GST trade name	gst.gov.in/services/searchtp	Free	Session	Per-state loop
DPIIT Startup India	startupindia.gov.in	Free	None	JSON XHR
FSSAI	foscoss.fssai.gov.in	Free	Session	F&B clients
RBI banks/NBFCs	rbi.org.in	Free (CSV)	None	Daily refresh
SEBI intermediaries	sebi.gov.in	Free (CSV)	None	Weekly refresh
IRDAI	irdai.gov.in	Free	None	Insurance only
Patent Office	search.ipindia.gov.in	Free	Session + CAPTCHA	Prior-use proof
Trademarkia (fallback)	trademarkia.com/api	Paid — USD 99/mo	API key	Backup only

7.2 Domains

Connector	URL	Free / Paid	Auth	Notes
WHOIS (raw)	whois.iana.org	Free	None	Port 43
RDAP	rdap.org	Free	None	JSON / modern
GoDaddy	developer.godaddy.com	Free dev / affiliate	Key + secret	Revenue share

Namecheap	namecheap.com/support/api	Free w/ \$50 balance	Key + IP whitelist	Cheap TLDs
WhoisXML	whoisxmlapi.com	500/mo free; USD 19+/mo	API key	Bulk WHOIS
Domainr	domainr.com/docs/api	Paid — USD 25/mo from 10k	RapidAPI key	Best UX
INRegistry	registry.in	Free WHOIS	None	.in / .co.in
Cloudflare Registrar	api.cloudflare.com	Free (at-cost domains)	API token	Limited TLDs

7.3 Social Handles

Connector	URL	Free / Paid	Auth	Notes
Instagram Graph	developers.facebook.com/docs/instagram-api	Free w/ Meta dev	OAuth 2.0	Public probe better
X (Twitter)	developer.x.com	Paid — USD 200/mo Basic	OAuth 2.0	Free tier 1500/mo only
YouTube Data v3	developers.google.com/youtube/v3	Free (10k units/day)	API key	Cache aggressively
LinkedIn	developer.linkedin.com	Restricted partner	OAuth 2.0	Use URL probe
Facebook Graph	developers.facebook.com	Free w/ Meta dev	OAuth 2.0	URL probe primary
Threads	developers.facebook.com/docs/threads	Free preview	OAuth 2.0	URL probe fallback
Telegram Bot API	core.telegram.org/bots/api	Free	Bot token	getChat method
WhatsApp Business	business.whatsapp.com	Paid per conversation	OAuth + verify	Display-name check
Pinterest	developers.pinterest.com	Free	OAuth 2.0	Business profile slug

7.4 Marketplaces & Stores

Connector	URL	Free / Paid	Auth	Notes
google-play-scraper	github.com/facundoolano/google-play-scraper	Free OSS	None	npm package

iTunes Search	itunes.apple.com/search	Free	None	Public endpoint
Product Hunt v2	api.producthunt.com/v2	Free w/ OAuth	OAuth 2.0	GraphQL
GitHub REST	docs.github.com/en/rest	Free (5k/hr auth)	PAT	GET /users/<name>
Shopify probe	shopify.dev/api	Free (DNS)	None	DNS resolve
Amazon (3rd-party)	scrapingbee.com	Paid — USD 49+/mo	Provider key	Heavy bot detect
Flipkart (3rd-party)	brightdata.com	Paid — USD 500+/mo	Provider key	Residential proxies

7.5 Brand Collision & SEO

Connector	URL	Free / Paid	Auth	Notes
Google CSE	developers.google.com/custom-search	100/day free	API key + CX	Filter gl=in
SerpAPI	serpapi.com	Paid — USD 50/mo from 5k	API key	Full SERP
ScrapingBee	scrapingbee.com	1k free credits; USD 49+/mo	API key	Dynamic pages
Bright Data	brightdata.com	Paid — USD 500+/mo	Account + zone	Residential
Wikipedia API	en.wikipedia.org/w/api.php	Free	None	Concept-page check
Wikidata SPARQL	query.wikidata.org/sparql	Free	None	Disambiguation
pytrends	github.com/GeneralMills/pytrends	Free OSS	None	Unofficial — risky
Crunchbase Basic	data.crunchbase.com	Paid — USD 49+/mo	API key	Funded startups

7.6 Linguistic, Transliteration & Numerology

Connector	URL	Free / Paid	Auth	Notes
Bhashini (Govt)	bhashini.gov.in/ulca	Free (registered)	Account token	22 languages
Indic NLP Library	github.com/	Free OSS	None	Self-host

	anoopkunchukuttan/ indic_nlp_library			
Aksharamukha	aksharamukha.appspot.com	Free	None	ISO 15919
Google Translate	cloud.google.com/ translate	500k chars/mo free	API key + GCP	Fallback only
ICU4J / PyICU	icu.unicode.org	Free OSS	None	Unicode normalisation
Chaldean engine	internal://numerology	Proprietary	None	Pure Python

7.7 Infrastructure — CAPTCHA · proxies · queues · real-time

Connector	URL	Free / Paid	Auth	Notes
2Captcha	2captcha.com	USD 2.99/1k	API key	Deep tier only
Anti-Captcha	anti-captcha.com	USD 2.00/1k	API key	Failover
CapMonster Cloud	capmonster.cloud	USD 0.50–1.50/1k	API key	Cheapest
Bright Data proxies	brightdata.com	USD 15.75/GB	Zone token	Hardest sources
Smartproxy	smartproxy.com	USD 7/GB starter	IP whitelist	Cheaper residential
Redis	redis.io / upstash.com	Free OSS / USD 7+/mo managed	Self-host	Core dependency
BullMQ	docs.bullmq.io	Free OSS	Same as Redis	Job queue
Socket.IO	socket.io	Free OSS	Token / session	Self-host
Pusher Channels	pusher.com/channels	200k/day free; USD 49+/mo	App key + secret	Managed alt
AWS WS API Gateway	aws.amazon.com/api-gateway	USD 1/1M msgs	IAM	Scale-cheap
Cloudflare R2	cloudflare.com/ products/r2	10GB free; USD 0.015/GB	API token	Zero egress
AWS S3	aws.amazon.com/s3	Free 5GB/12mo; USD 0.023/GB	IAM	Pre-signed URLs

7.8 Authentication & Payments

Connector	URL	Free / Paid	Auth	Notes
Firebase Phone Auth	firebase.google.com/	Free 10k/mo; USD	Firebase config	Global OTP

	docs/auth/web/phone-auth	0.01–0.06/SMS		
MSG91 (India SMS)	msg91.com	100 trial; ₹0.18–0.25/SMS	Auth key	DLT-registered
WhatsApp Business OTP	developers.facebook.com/docs/whatsapp	Free dev; ₹0.30–0.80/msg	Meta business	Preferred in India
Razorpay	razorpay.com/docs	2% / UPI 0%	Key + secret	India primary
Stripe	stripe.com/docs	2.9% + ₹3	Key + webhook secret	International
ClearTax e-invoice	cleartax.in	₹0.50–2.00/invoice	API key	GST automation

8. Rate Limiting, Resilience, Observability

8.1 Rate-limit strategy

- Per-source token-bucket in Redis. Refill rate calibrated to each source's documented limits (see \$7 column "Rate Limit").
- Per-IP scan limit for free tier: 60 scans / hour. Returns 429 with `Retry-After`.
- Per-user scan limit for retail: 5 deep scans / day (anti-abuse).
- Per-agency-org quota: configured per contract, enforced at JWT validation.

8.2 Retry policy

- Each scanner job retries up to 3 times with exponential backoff (1s, 4s, 16s).
- On final failure, `tile_id` is emitted with ``status: "error"``` and the verdict aggregator ignores it (does not score it as pass or fail).
- CAPTCHA failures trigger an automatic failover to the secondary solver.

8.3 Circuit breakers

- Each connector has a circuit breaker: 5 consecutive errors → open state for 60 seconds, during which the tile returns "Checking..." and never emits result.
- Breaker state surfaces on `/v1/healthz` as a JSON object listing open circuits.

8.4 Observability

- Sentry for errors. Tag every event with ``scan_id``, ``tile_id``, ``source_id``.
- Grafana Loki for logs. JSON structured logs with consistent field names.
- Prometheus for metrics. Key SLIs: `scan_p50_latency`, `scan_p95_latency`, `source_error_rate`, `queue_depth`, `captcha_solve_rate`.
- Better Stack for synthetic uptime probes against `/v1/healthz` from 5 regions.

8.5 Alerting

- P0 — `scan_failed` rate > 1% over 5min → PagerDuty.
- P1 — any source `error_rate` > 30% over 15min → Slack #ops-alerts.
- P2 — queue depth > 5000 over 10min → Slack #ops-alerts.
- P3 — daily summary email with top 10 noisy sources.

9. Security & Compliance

9.1 Data classification

- Public — anonymous scan inputs, public-register lookups (cached freely).
- Confidential — phone, email, account history (encrypted at rest, AES-256).
- Highly Confidential — payment tokens (never stored; provider tokenises).

9.2 DPDP Act compliance (Digital Personal Data Protection Act, 2023)

- Lawful purpose, consent, purpose-limitation, storage-limitation, security-safeguard obligations honoured.
- Section 5 notice and privacy policy published; consent captured on sign-up.
- Data Principal rights — access, correction, erasure, grievance redressal — implemented in account UI.
- DPO/grievance officer contact published in /privacy.
- Free tier collects only anonymous IP + name string; not personal data within the Section.

9.3 IT Act, 2000 — Section 65

Source code repositories are private (GitHub) with branch protections, signed commits, and audit logs. Source code is retained as long as the work is in service plus 3 years.

9.4 Scraping ethics

- Strict robots.txt observance for sources whose ToS prohibit automated access (we do NOT scrape them; we link the user out).
- Rate limits respected per source documentation.
- No login bypass, no CAPTCHA bypass against sources that disallow it.
- CAPTCHA solving used only against Government registers (MCA, IP India, Copyright) where Section 399 of the Companies Act, public register access under Trade Marks Act and Copyright Act expressly contemplate public read access.

9.5 Transport & storage

- TLS 1.3 across the board. HSTS, CSP, X-Frame-Options on all responses.
- PostgreSQL — at-rest AES-256 via cloud provider; row-level encryption for phone/email columns via pgcrypto.
- R2/S3 — server-side encryption with provider-managed keys.
- Redis — TLS-only, password-auth, ACLs per service.

9.6 Vulnerability management

- GitHub Dependabot + Snyk scanning on every PR.
- Quarterly penetration test (third-party).
- Bug bounty programme (hosted on Bugcrowd) — to be opened post-launch.

10. Deployment, CI/CD & Runbook

10.1 Environments

- development — local Docker Compose. All services + a mock orchestrator.
- staging — Hetzner CCX11 single-node Kubernetes (k3s). Deployed on every merge to `main`.
- production — multi-zone deployment as in §2.4. Deployed via ArgoCD on manual tag promotion.

10.2 CI/CD pipeline

- GitHub Actions runs: lint, unit tests, integration tests, type-check, build Docker images, push to registry.
- ArgoCD picks up tagged images and syncs to k8s manifests.
- Database migrations via Alembic (Python) / Prisma (Node) — run in pre-deploy hook with idempotency guard.

10.3 On-call runbook

- Primary on-call rotation: weekly, Mon-Sun.
- Secondary: same week, escalation after 15min.
- Common incidents — runbook entries: (a) MCA portal down, (b) Captcha provider quota exceeded, (c) Queue depth runaway, (d) WS server pod restarts, (e) Cloudflare edge incident.

11. Code Examples (selected)

11.1 Submitting a scan (front-end, TypeScript)

```
import { io, Socket } from "socket.io-client";

interface ScanResponse {
  scan_id: string;
  websocket_url: string;
  eta_seconds: number;
}

export async function runScan(name: string, mode: "business"|"baby") {
  const res = await fetch("/api/v1/scan", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name, mode, tier: "free" }),
  });
  if (!res.ok) throw new Error(`Scan failed: ${res.status}`);
  const { scan_id, websocket_url }: ScanResponse = await res.json();

  const socket: Socket = io(websocket_url, { transports: ["websocket"] });
  socket.on("result_event", (t) => upsertTile(t));
  socket.on("hud_update", (h) => updateHud(h));
  socket.on("verdict_complete", (v) => finishScan(v));
  socket.on("tile_error", (e) => markTileError(e));
}
```

11.2 Domain availability via RDAP (worker, Node.js)

```
import fetch from "node-fetch";

export async function checkDomain(domain: string): Promise<TileResult> {
  const url = `https://rdap.org/domain/${domain}`;
  const res = await fetch(url, { headers: { "Accept": "application/rdap+json" } });

  if (res.status === 404) {
    return { tile_id: `dom_${domain}`, status: "ok", summary: "Available" };
  }
  if (res.status === 200) {
    const body = await res.json();
    const expires = body.events?.find(e => e.eventAction === "expiration")?.eventDate;
    return { tile_id: `dom_${domain}`, status: "no",
      summary: "Taken", detail: { expires } };
  }
  throw new Error(`RDAP returned ${res.status}`);
}
```

11.3 Chaldean numerology engine (Python, the entire algorithm)

```

CHALDEAN_MAP = {
    **dict.fromkeys("AIJQY", 1), **dict.fromkeys("BKR", 2),
    **dict.fromkeys("CGLS", 3), **dict.fromkeys("DMT", 4),
    **dict.fromkeys("EHNX", 5), **dict.fromkeys("UVW", 6),
    **dict.fromkeys("OZ", 7), **dict.fromkeys("FP", 8),
}
RULING_PLANET = {1:"Sun",2:"Moon",3:"Jupiter",4:"Uranus",5:"Mercury",
    6:"Venus",7:"Neptune",8:"Saturn",9:"Mars"}
INDUSTRY_FIT = {
    5: {"good": ["Media", "SaaS", "Tech", "Mobility", "Education", "Comms"],
        "avoid": ["Banking", "Insurance", "Real estate"]},
    # ... other root numbers
}

def chaldean(name: str) -> dict:
    letters = [c for c in name.upper() if c.isalpha()]
    digits = [CHALDEAN_MAP[c] for c in letters if c in CHALDEAN_MAP]
    compound = sum(digits)
    root = compound
    while root > 9:
        root = sum(int(d) for d in str(root))
    return {
        "letters": letters,
        "digits": digits,
        "compound": compound,
        "root": root,
        "planet": RULING_PLANET[root],
        "fit": INDUSTRY_FIT.get(root, {"good": [], "avoid": []}),
    }

```

11.4 WebSocket event emission (server, Node.js)

```

import { Server } from "socket.io";
import { createAdapter } from "@socket.io/redis-adapter";
import { Redis } from "ioredis";

const pub = new Redis(REDIS_URL);
const sub = pub.duplicate();
const io = new Server({ cors: { origin: "*" } });
io.adapter(createAdapter(pub, sub));

io.of("/v1/stream").on("connection", (socket) => {
    const { scanId } = socket.handshake.query;
    socket.join(`scan:${scanId}`);
    replayBuffer(scanId).then(events => events.forEach(e => socket.emit(e.type, e.data)));
});

// from any worker:
export function emitTile(scanId: string, tile: TileResult) {
    io.of("/v1/stream").to(`scan:${scanId}`).emit("result_event", tile);
}

```

```
    appendToReplayBuffer(scanId, "result_event", tile);  
}
```

12. Appendices

Appendix A — Paired Excel deliverable

The full connector knowledge base, scanner ownership matrix, and live cost model are in the companion file **Naam_Dekho_Connector_Catalog.xlsx** delivered alongside this document. Three sheets: (1) Connector Catalog — every connector with URL, free/paid status, rate limit, auth, notes; (2) Scanner Modules — worker pool sizes, response times, cache TTLs, owning squad; (3) Cost Model — fully-formula-driven per-scan cost analysis where you can change the blue input cells and watch the bottom-line gross margin recompute live.

Appendix B — Related diagrams

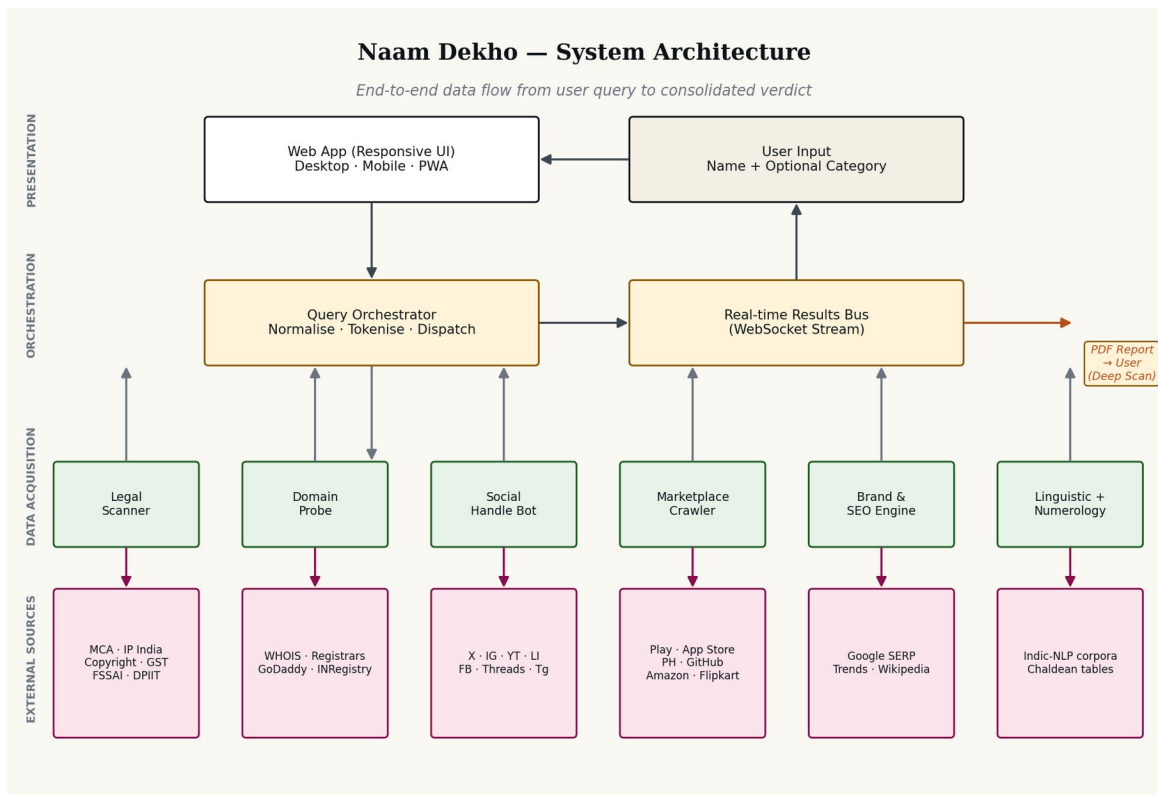


Figure A1 — System architecture — layered view (recap)

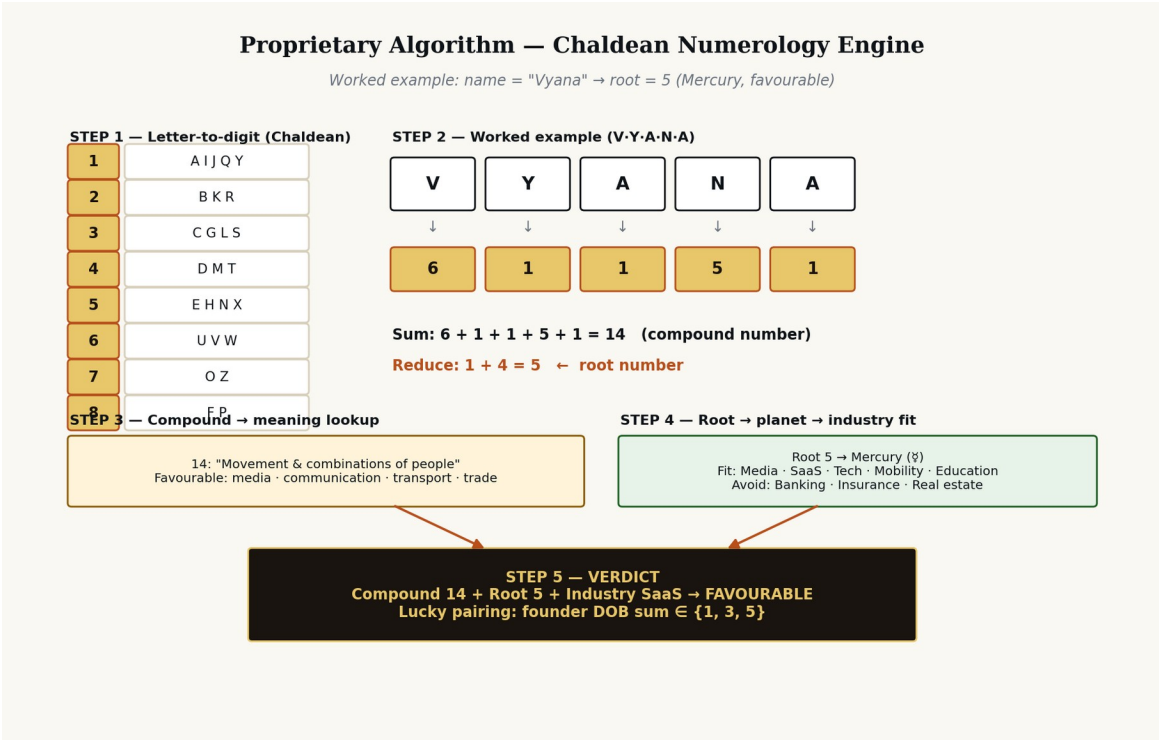


Figure A2 — Chaldean numerology engine — implementation algorithm

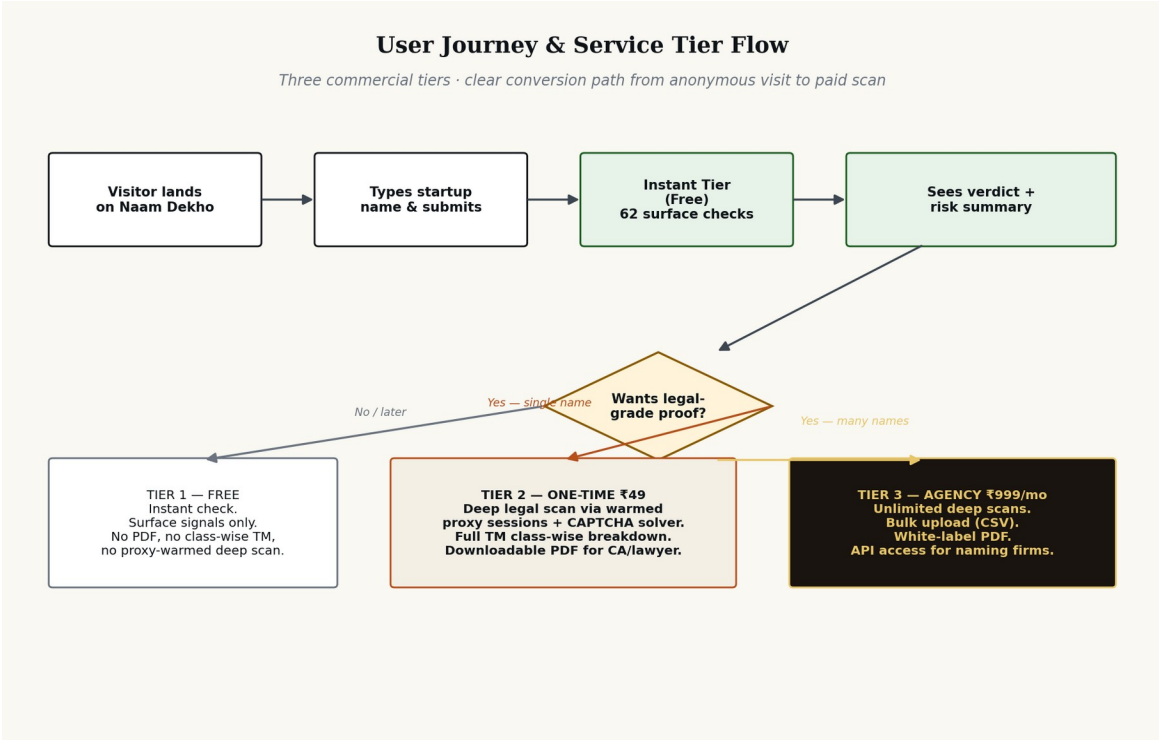


Figure A3 — User journey & three-tier service model

Appendix C — Glossary

- BullMQ — Redis-backed job queue library for Node.js. Used as the task queue.
- CSE — Custom Search Engine. Google's programmable web-search product.
- DPDP Act — Digital Personal Data Protection Act, 2023 (India).
- MCA21 — The portal at mca.gov.in for Companies Act 2013 filings and lookups.
- RDAP — Registration Data Access Protocol, the JSON successor to WHOIS.
- Tile — One platform's result in the result page UI; corresponds 1:1 with a result_event over WebSocket.
- ULID — Universally Unique Lexicographically Sortable Identifier. Used for scan_id.

Appendix D — Open questions for the team

- Confirm primary infra region: AWS Mumbai vs Hetzner Frankfurt (latency vs cost).
- Decide on SMS provider final choice between MSG91, Firebase, and WhatsApp Business OTP.
- Build vs buy on PDF generation — WeasyPrint vs ReportLab vs Browserless.
- Sandbox vs live integration with Bhashini (their stability is still maturing as of Q2 2026).
- Per-organisation analytics dashboard for agency tier — Phase 2 scope.
- Disaster recovery — RPO/RTO targets for production data.

— End of Developer Documentation —